AUTOMATION & QA DEVELOPER

Task 1 – Web App QA Report

RealWorld / Conduit exploratory testing worksheet

Candidate	Aditya Javalekar
Application	RealWorld – Conduit (demo application)
Test environment	Browser-based public demo; browser/version and test date to be recorded during execution
Report status	**Draft – execution evidence required**

Important integrity note: This is a prepared QA worksheet, not a claim that the live application has already been manually tested. Candidate defects below are hypotheses/test targets. Record an issue as a confirmed bug only after reproducing it and capturing the actual result.

1. Objective and scope

Assess the core Conduit user journeys and identify functional, validation, usability, reliability, and security concerns. The assessment brief asks for at least five issues and one 5–10 sentence root-cause analysis. This draft defines the test coverage and evidence fields needed to complete that deliverable.

2. Test coverage

Area	Test activity	Evidence to capture
Registration	Create a new account; retry with the same email/username; submit missing fields.	Form state, response/message, whether duplicate account is blocked.
Login / logout	Login with valid and invalid credentials; log out; revisit protected pages.	Success/error message, redirect, session/token behavior.
Create article	Publish a valid article; submit with missing title, description, or body.	Validation feedback, saved article, network response if available.
Edit article	Change title/body, save, refresh, and reopen the article.	Updated values before and after refresh.
Delete article	Delete an owned article; revisit its URL and article list.	Confirmation behavior, list state, resulting page/status.
Edge cases	Try long text, special characters, repeated clicks, and network interruption.	Loading state, validation, duplicate actions, recovery behavior.

3. Bug report register

The five entries below are investigation targets, not confirmed findings. Complete the Actual Result and Severity only after execution. If behavior matches expectation, mark the test Pass and replace it with another reproducible issue.

#	Candidate issue / test	Steps to reproduce	Expected vs actual / status	Severity	Suspected cause
1	Duplicate account validation	Register a test user; submit registration again using the same email or username.	Expected: duplicate is rejected with clear feedback. Actual: record observed during test. Status: NOT RUN.	TBD	Missing/incorrect uniqueness validation or unclear UI error mapping.
2	Invalid login feedback	Attempt login with a valid email and incorrect password.	Expected: safe, clear error; no authenticated session. Actual: record during test. Status: NOT RUN.	TBD	Error response not handled or session state incorrectly updated.
3	Required article fields	Open editor and submit with title, description, or body blank, one at a time.	Expected: submission blocked with field-level feedback. Actual: record during test. Status: NOT RUN.	TBD	Client/server validation gap or missing error display.
4	Edit persistence	Create article; edit title/body; save; refresh and reopen.	Expected: saved values persist after refresh. Actual: record during test. Status: NOT RUN.	TBD	Update request, stale client state, or cache/refresh issue.
5	Delete consistency	Delete an owned article; revisit its URL and refresh the article list.	Expected: article is absent and direct URL returns not found. Actual: record during test. Status: NOT RUN.	TBD	Delete request failure, stale cache, or route state not refreshed.

Severity guide

Critical: data loss, account compromise, or complete service outage. **High:** major core flow blocked or serious security exposure. **Medium:** feature works incorrectly with a workaround. **Low:** minor usability or cosmetic issue.

4. Root-cause analysis – template

Selected issue: [insert one confirmed issue and ID].

1. Describe the exact user action and the observed failure. 2. State the expected behavior and the actual behavior, including any visible message or HTTP status. 3. Identify the component most likely responsible (form validation, frontend state, API request, authorization, or persistence). 4. Explain the evidence supporting that hypothesis. 5. Describe how to reproduce the issue consistently. 6. Propose a fix at the correct layer, not only a cosmetic workaround. 7. Add a regression test for the failing scenario. 8. Retest the original flow and adjacent edge cases after the fix.

5. Execution record to complete

Field	Value
Test date/time	[Enter date and local time]
Browser / version / OS	[Enter actual environment]
Application URL	https://demo.realworld.io
Test account	[Use a dedicated test account; do not include its password in this report]
Screenshots / recordings	[Attach evidence filenames and reference each issue]
Final result	[Number of confirmed issues; passed/failed/blocked test cases]

Reference: RealWorld frontend API documentation describes the official demo API and notes content-visibility restrictions for logged-out and logged-in users. This matters when interpreting article-list results during testing. See https://docs.realworld.show/specifications/frontend/api/ .